

```
"""
```

```
=====
ANÁLISE DE TRELIÇAS PLANAS ISOSTÁTICAS
VERSÃO CORRIGIDA
=====
```

Correções realizadas:

- Ajuste no número de equações/incógnitas
- Correção da montagem da matriz
- Correção das reações de apoio
- Correção do método de Gauss
- Correção dos gráficos

Requisitos:

pip install matplotlib

```
=====
"""
```

```
import math
import matplotlib.pyplot as plt
```

```
# =====
# MÉTODO DE GAUSS (MANUAL)
# =====
```

```
def gauss(A, b):
```

```
    n = len(b)
```

```
    # matriz aumentada
    M = []
```

```
    for i in range(n):
        M.append(A[i][:] + [b[i]])
```

```
    # eliminação
    for k in range(n):
```

```
        # pivotamento parcial
        max_row = k
```

```
        for i in range(k + 1, n):
            if abs(M[i][k]) > abs(M[max_row][k]):
                max_row = i
```

```
        M[k], M[max_row] = M[max_row], M[k]
```

```

pivot = M[k][k]

if abs(pivot) < 1e-12:
    raise ValueError("Sistema singular!")

# normalização
for j in range(k, n + 1):
    M[k][j] /= pivot

# eliminação
for i in range(k + 1, n):

    factor = M[i][k]

    for j in range(k, n + 1):
        M[i][j] -= factor * M[k][j]

# retrossubstituição
x = [0.0] * n

for i in range(n - 1, -1, -1):

    x[i] = M[i][n]

    for j in range(i + 1, n):
        x[i] -= M[i][j] * x[j]

return x

# =====
# DADOS DA TRELIÇA
# =====

# Coordenadas dos nós
x = [0, 2, 1, 3]
y = [0, 0, 1, 1]

N = len(x)

# Barras
bars = [
    (0, 1),
    (0, 2),
    (1, 2),
    (1, 3),
    (2, 3)
]

```

```

B = len(bars)

# Carregamentos
Fx = [0, 0, 0, 0]
Fy = [0, 0, -1000, 0]

# =====
# INCÓGNITAS
# =====

# 5 forças internas + 3 reações
unknowns = B + 3

# 2N equações
eqs = 2 * N

A = [[0.0 for _ in range(unknowns)] for _ in range(eqs)]
b = [0.0 for _ in range(eqs)]

# =====
# MONTAGEM DAS BARRAS
# =====

for k, (i, j) in enumerate(bars):

    dx = x[j] - x[i]
    dy = y[j] - y[i]

    L = math.sqrt(dx**2 + dy**2)

    cx = dx / L
    cy = dy / L

    # Equilíbrio em X
    A[2 * i][k] += cx
    A[2 * j][k] -= cx

    # Equilíbrio em Y
    A[2 * i + 1][k] += cy
    A[2 * j + 1][k] -= cy

# =====
# REAÇÕES DE APOIO
# =====

```

```
# Nó 0 -> apoio fixo
```

```
A[0][B + 0] = 1 # Rx0
```

```
A[1][B + 1] = 1 # Ry0
```

```
# Nó 1 -> apoio móvel vertical
```

```
A[3][B + 2] = 1 # Ry1
```

```
# =====
```

```
# CARGAS EXTERNAS
```

```
# =====
```

```
for i in range(N):
```

```
    b[2 * i] = -Fx[i]
```

```
    b[2 * i + 1] = -Fy[i]
```

```
# =====
```

```
# RESOLVER SISTEMA
```

```
# =====
```

```
solution = gauss(A, b)
```

```
forces = solution[:B]
```

```
Rx0 = solution[B + 0]
```

```
Ry0 = solution[B + 1]
```

```
Ry1 = solution[B + 2]
```

```
# =====
```

```
# RESULTADOS
```

```
# =====
```

```
print("\n===== RESULTADOS =====\n")
```

```
for k, f in enumerate(forces):
```

```
    if abs(f) < 1e-6:
```

```
        estado = "NULA"
```

```
    elif f > 0:
```

```
        estado = "TRAÇÃO"
```

```
    else:
```

```
        estado = "COMPRESSÃO"
```

```

print(f"Barra {k}: {f:.2f} N -> {estado}")

print("\n===== REAÇÕES =====\n")

print(f"Rx0 = {Rx0:.2f} N")
print(f"Ry0 = {Ry0:.2f} N")
print(f"Ry1 = {Ry1:.2f} N")


# =====
# ESTATÍSTICAS
# =====

max_force = max(forces, key=abs)
media = sum(abs(f) for f in forces) / B

print("\n===== ESTATÍSTICAS =====\n")

print(f"Força máxima: {max_force:.2f} N")
print(f"Força média : {media:.2f} N")


# =====
# PLOT 1 - TRELIÇA ORIGINAL
# =====

plt.figure(figsize=(8, 6))

for (i, j) in bars:

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        'ko-',
        linewidth=2
    )

# nós
for i in range(N):
    plt.text(x[i], y[i] + 0.05, f'N{i}', fontsize=12)

plt.title("Treliza Original")
plt.axis('equal')
plt.grid(True)


# =====
# PLOT 2 - MAPA DE CALOR

```

```
# =====

plt.figure(figsize=(8, 6))

max_abs = max(abs(f) for f in forces)

for k, (i, j) in enumerate(bars):

    f = forces[k]

    intensidade = abs(f) / max_abs

    if f >= 0:
        cor = (0, 0, intensidade)
    else:
        cor = (intensidade, 0, 0)

    plt.plot(
        [x[i], x[j]],
        [y[i], y[j]],
        color=cor,
        linewidth=4
    )

plt.title("Mapa de Calor")
plt.axis('equal')
plt.grid(True)
```

```
# =====
# PLOT 3 - GRÁFICO DE BARRAS
# =====
```

```
plt.figure(figsize=(8, 5))

nomes = [f'B{k}' for k in range(B)]

cores = []

for f in forces:
    if f >= 0:
        cores.append("blue")
    else:
        cores.append("red")

plt.bar(nomes, forces, color=cores)

plt.title("Forças nas Barras")
```

```
plt.ylabel("Força (N)")
plt.grid(True)
```

```
# =====
# PLOT 4 - HISTOGRAMA
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
plt.hist(forces, bins=5, color='gray', edgecolor='black')
```

```
plt.title("Histograma das Forças")
plt.xlabel("Força (N)")
plt.ylabel("Frequência")
plt.grid(True)
```

```
# =====
# EXIBIR
# =====
```

```
plt.show()
```